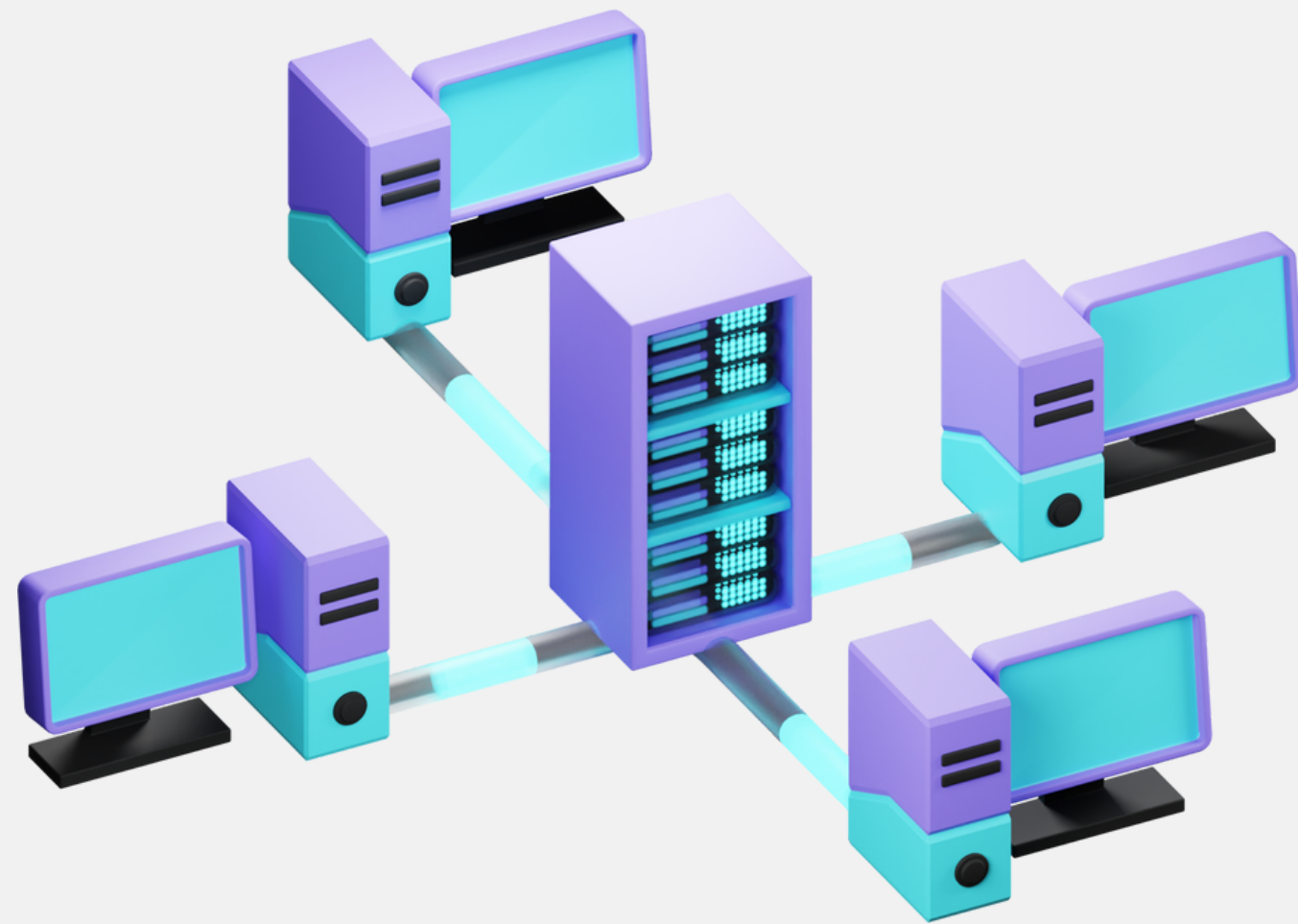
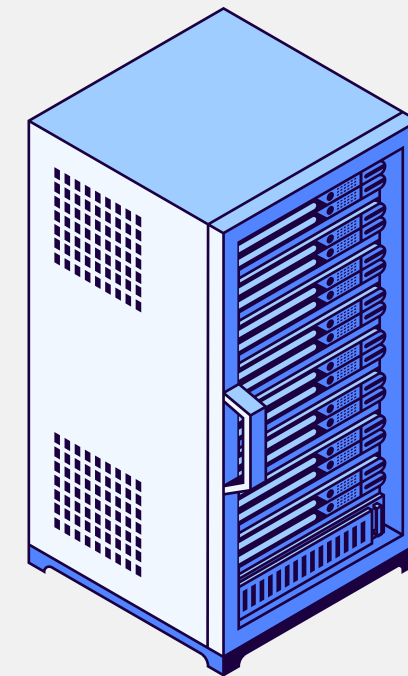




Modello Client/Server e protocollo applicativo HTTP

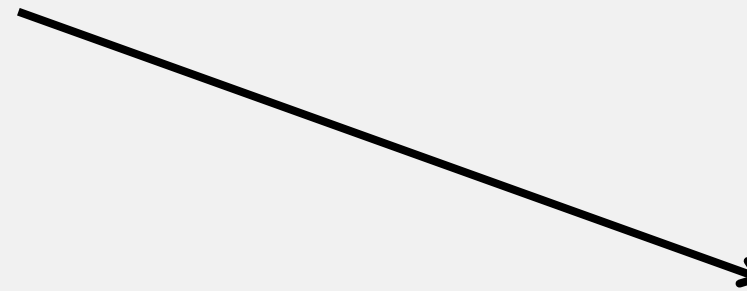
Protocolli applicativi che usano TCP prevedono questa struttura:



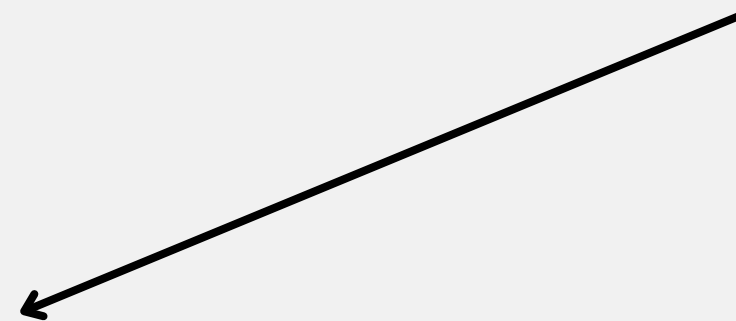
Client: invia una richiesta ad un server di cui conosce l'indirizzo IP e il numero di porta del servizio desiderato

Server: ospita un processo sempre attivo e pronto a ricevere richieste, su una porta nota

**Il client contatta
quindi un server
attraverso indirizzo IP
e porta del servizio**



**Il server elaborerà la
richiesta del client**



**Il client riceverà una
risposta, positiva o
negativa**

Classi e metodi utili

Connessione TCP

Lato Server

- **ServerSocket (classe):** Ascolta le connessioni in entrata.
 - **ServerSocket(int port):** Crea un server socket su una porta specifica.
 - **accept():** Aspetta una connessione da parte di un client.
- **Socket (classe):** Rappresenta la connessione con il client.
 - **getInputStream():** Restituisce un InputStream per ricevere dati dal client.
 - **getOutputStream():** Restituisce un OutputStream per inviare dati al client.

```
public static void main(String[] args) throws Exception{
    //dichiaro numero di porta del server, la server socket
    //e la socket per il trasferimento dati
    int porta = 3333;
    ServerSocket server;
    Socket s_client;

    //Scanner s = new Scanner(System.in);
    int conta_richieste=0;
    //dichiaro il mio flusso di output e
    OutputStream outS;
    PrintWriter pr;
```

Inizializzazione ServerSocket

Attesa di un client. Quando c'è un contatto, si sblocca e restituisce una Socket

Si ottiene il flusso di byte in Output

L'output viene gestito con i caratteri True: autoflush in funzione. Non serve la funzione flush per inviare

```
try{
    //inizializzo la mia serverSocket mettendomi in ascolto sulla porta
    server = new ServerSocket(port:porta);
    System.out.println("Server attivo. Ascolto sulla porta "+porta);
    System.out.println("In attesa di connessione...");
    //finchè un client non contatta la mia porta, rimani in ascolto. Quando un client
    //contatterà il server, il metodo accept ritorna una socket da assegnare alla mia variabile socket
    s_client = server.accept();

    conta_richieste++;
    System.out.println("Connesso con un client. Richiesta numero: "+conta_richieste);

    //dalla socket ottengo l'outputStream col metodo getOutputStream
    outS = s_client.getOutputStream();
    //per inviare caratteri, uso la classe PrintWriter. Il true nel costruttore è
    //per l'autoflush, ovvero non serve il flush() per inviare effettivamente il messaggio
    pr = new PrintWriter(out: outS, autoFlush: true);

    Date oggi = new Date();
    String data_oggi = oggi.toString();
    pr.println("Data di oggi: "+data_oggi);

    pr.close();
    s_client.close();
}
```

Invia il contenuto inserito nel metodo

Classi e metodi utili

Lato Client

- **Socket (classe):** Rappresenta la connessione con il server.
 - **getInputStream():** Restituisce un **InputStream** per ricevere dati dal client.
 - **getOutputStream():** Restituisce un **OutputStream** per inviare dati al client.

L'asimmetria client-server è notevole. Inoltre, il costruttore di Socket deve ricevere in input un indirizzo IP e un numero di Porta (int).

Quando creo la Socket del client in questo modo, sto contattando direttamente il Server, che se in attesa e disponibile, instaurerà una connessione affidabile

Dichiaro la mia Socket

Dichiaro un InputStream per avere il flusso di input. InputStreamReader mi trasforma tutto in caratteri. BufferedReader serve per una lettura più efficiente

Inizializzando la Socket, contatto il server. Il costruttore prende in input l'indirizzo IP e la Porta (dell'applicazione) del Server

in è il flusso di byte. Questo viene trasformato in flusso di byte con InputStreamReader. BufferedReader utilizza i metodi come read o readLine per leggere i caratteri in modo efficiente.

```
public static void main(String[] args) throws IOException{

    //dichiaro la socket e la porta che andrò a contattare
    Socket client;
    int porta = 3333;

    Scanner s = new Scanner( source: System.in);

    //dichiaro il mio InputStream ovvero il flusso di input, che arriverà dal server.
    //InputStreamReader mi converte il flusso di byte in caratteri
    InputStream in;
    InputStreamReader indata;
    BufferedReader bin;
    try{

        System.out.println( «: "Sto cercando di connettermi..."");
        //inizializzo la mia socket. In questo modo, contatto il server
        client = new Socket( address: InetAddress.getLocalHost(), port: porta);
        System.out.println("Connesso alla porta numero: "+porta);

        //la gestione di un input di caratteri è così.
        in = client.getInputStream();
        indata = new InputStreamReader(in);
        bin = new BufferedReader( in: indata);

        //leggo ciò che ho ricevuto
        String clock = bin.readLine();
        System.out.println( «: clock);
        bin.close();
        client.close();
    }
}
```

Classi e metodi utili

INPUT:

InputStream: Classe astratta.
Gestisce il flusso di input di byte. Si ottiene dal metodo `getInputStream` della classe `Socket`.

InputStreamReader: Classe.
Permette di leggere byte e decodificarli in caratteri usando una specifica codifica

BufferedReader: Classe. Fornisce buffering per la lettura efficiente di caratteri, array e linee.
Metodi: `read()`, `readLine()`.

OUTPUT

OutputStream: Classe astratta.
Gestisce il flusso di output di byte. Si ottiene dal metodo `getOutputStream` della classe `Socket`. Metodi per inviare dati:

PrintWriter: Classe. Il suo costruttore riceve in input `OutputStream`. Si può aggiungere un `true` per fissare l'autoflush. Permette di inviare caratteri attraverso i metodi `print`, `println`. Concatenando `PrintWriter` e `OutputStream` viene già gestito la conversione da caratteri in byte.

InputStream:

- **int read():** Legge il prossimo byte di dati dal flusso di input. Restituisce il byte letto come un intero nel range 0-255, o -1 se la fine del flusso è stata raggiunta.
- **int read(byte[] b):** Legge alcuni byte di dati e li memorizza nell'array b. Restituisce il numero di byte effettivamente letti, o -1 se la fine del flusso è stata raggiunta.
- **int read(byte[] b, int off, int len)** Legge fino a len byte di dati e li memorizza nell'array b, iniziando dall'indice off. Restituisce il numero di byte letti, o -1 se la fine del flusso è stata raggiunta.
- **void close().** Chiude lo stream di input e rilascia tutte le risorse associate.

OutputStream:

- **void write(int b)** Scrive il byte specificato nel flusso di output. Il byte è specificato come un intero, ma solo i 8 bit meno significativi vengono scritti.
- **void write(byte[] b)** Scrive b.length byte dall'array di byte b nello stream di output.
- **void write(byte[] b, int off, int len)** Scrive len byte dall'array di byte b, a partire dall'indice off, nello stream di output.
- **void flush()** Scarica il buffer di output, inviando tutti i dati accumulati al destinatario.
- **void close()** Chiude lo stream di output e rilascia tutte le risorse associate.

Quando si utilizzano i byte, in input e in output si può utilizzare un buffer. Le classi sono rispettivamente `BufferedInputStream` e `BufferOutputStream`

Da notare che entrambe le classi possono ricevere in input nel loro costruttore uno stream di byte, che sia una connessione con un altro dispositivo, sia nel contesto dei file

InetAddress

- **Classe:** Rappresenta un indirizzo IP.
- **Metodi:** `getByName()`, `getLocalHost()`, `getHostAddress()`.
- **Utilizzo tipico:** Gestione e risoluzione degli indirizzi IP.

- **`getByName(String host)`:** Risolve il nome dell'host in un indirizzo `InetAddress`.
- **`getLocalHost()`:** Ritorna l'indirizzo `InetAddress` dell'host locale.
- **`getHostAddress()`:** Ritorna l'indirizzo IP sotto forma di stringa.

- **`Socket.getRemoteSocketAddress()`:** Ritorna l'indirizzo remoto a cui è connesso il socket. (IP:Porta)

Un server multicient deve poter svolgere il lavoro richiesto mentre è in attesa di altre connessioni

→ **La classe Server attenderà con la sua ServerSocket un nuovo cliente**

→ **Un Thread Worker eseguirà il compito richiesto**

Tutto in un ciclo infinito, in quanto il server è sempre in ascolto

Dichiaro la mia Socket

```
public static void main(String[] args) throws Exception {
```

```
    ServerSocket ss = new ServerSocket( port: 7000);
```

```
    while(true) {
```

```
        System.out.println( "Server in attesa di connessioni ...");
```

Finché un nuovo client non si collega, rimango in attesa.

```
        Socket s = ss.accept();
```

```
        Worker w;
```

```
        w = new Worker( socket: s);
```

```
        w.start();
```

Queste righe di codice vengono eseguite se un client ha contattato il server. Parte un Thread dedicato con la Socket connessa a quel client

Il ciclo quindi ripartirà. Mentre uno o più Thread sono in funzione, il server tornerà alla ServerSocket, attendendo un client nuovo

```
    }
```



```
public static class Worker extends Thread {
    private Socket clientSocket;
    private final String fileToSend = "C:\\Users\\utente\\Desktop\\Uscita 19-20 Novembre/3f85daac-7f5f-4c32-b9

    public Worker(Socket socket) {
        this.clientSocket = socket;
    }

    @Override
    public void run() {
        try {
            File file = new File( pathname: fileToSend);
            FileInputStream fileIn = new FileInputStream(file);
            BufferedInputStream in = new BufferedInputStream( in: fileIn);
            BufferedOutputStream out = new BufferedOutputStream( out: clientSocket.getOutputStream());

            byte[] buffer = new byte[4096];
            int bytesRead;
            while ((bytesRead = in.read( b: buffer)) != -1) {
                out.write( b: buffer, off: 0, len: bytesRead);
                out.flush();
            }
            fileIn.close();
            out.close();
            clientSocket.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Connessione UDP

Server

- **int port;**
- **DatagramSocket ds;**

Client

- **int serverPort**
- **InetAddress serverAddress**
- **DatagramSocket ds;**

Le Socket sono utilizzate per inviare e ricevere dati. Non esiste il concetto di ServerSocket, perché è la velocità il punto chiave, non più l'affidabilità.

```
public class Server extends Thread{

    private DatagramSocket ds;
    private int porta;

    public Server(int porta) throws SocketException{
        this.porta = porta;
        this.ds = new DatagramSocket( port: this.porta);

        // ds.setSoTimeout(10_000);
    }
```

```
public class Client {

    //Dichiaro le mie varibili fondamentali, soprattutto DatagramSocket
    private DatagramSocket ds;
    private InetAddress serverAddress;
    private int serverPort;

    public Client(String serverAddress, int serverPort) throws SocketException, UnknownHostException {

        //costruttore
        this.ds = new DatagramSocket();
        this.serverAddress = InetAddress.getByName( host: serverAddress);
        this.serverPort = serverPort;
        ds.setSoTimeout( timeout: 10_000); // Timeout dopo 10 secondi
    }
```


Connessione UDP

Ci si aspetta però che a fare la prima richiesta sia sempre il Client. Perciò lui dovrà avere anche IP e numero di porta del server da contattare. Come fa? Mandandogli direttamente il datagram

Trasmissione

```
String str = "Ciao server";  
byte[] bufSend = str.getBytes();
```

trasformo tutto in bytes



il datagram necessita del messaggio, della lunghezza del messaggio, dell'indirizzo e porta del destinatario



```
DatagramPacket dpSend = new DatagramPacket(bufSend,  
bufSend.length, serverAddress, serverPort);
```

```
ds.send(dpSend);
```

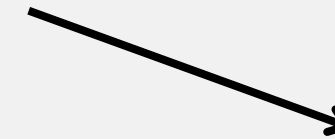
← Invio il mio datagram

Connessione UDP

Il server avrà ricevuto il messaggio se era in fase di ascolto

Ricezione

il datagram necessita del messaggio e della sua lunghezza. Non sa chi lo invierà



```
DatagramPacket dpReceive = new DatagramPacket(bufSend,  
bufSend.length);
```

```
ds.receive(dpReceive); ← attende fino a quando non riceve  
qualcosa
```

dpReceive ha al suo interno l'indirizzo e la porta del mittente. Usiamo alcuni metodi per spaccettare il nostro datagram

Connessione UDP

```
mitAdd = dpReceive.getAddress();  
mitPort = dpReceive.getPort();
```

ottengo IP e Porta del mittente



```
buf[] = dpReceive.getData()  
length = dpReceive.getLength()
```

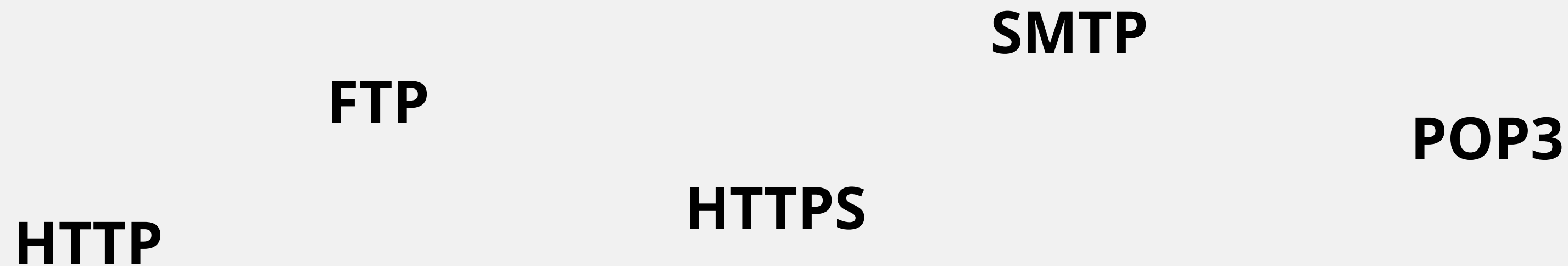
ottengo i dati del datagram e la loro lunghezza



Quando non è più necessaria la connessione, per esempio al lato client, chiudere la socket

```
ds.close();
```

Che tipo di servizi? Protocolli applicativi



Funzioni del livello applicazione: Questo livello fornisce servizi per le applicazioni software per eseguire operazioni di rete. È il livello più vicino all'utente finale, gestendo l'interazione specifica dei dati per le applicazioni.

HTTP

Definizione di HTTP (Hypertext Transfer Protocol): HTTP è il protocollo standard utilizzato per la trasmissione di documenti ipertestuali sul Web, come pagine HTML.

Funzionamento di base: In HTTP, un client (ad esempio, un browser) invia una richiesta a un server (sito web) che poi risponde con il contenuto richiesto. Questo scambio è fondamentale per il funzionamento del web.

Metodi più comuni

- **GET:** Richiede la risorsa identificata dall'URL
- **HEAD:** come GET ma ottiene solo alcune informazioni (header)
- **POST:** Integra la risorsa identificata come URL (non idempotente)
- **PUT:** Crea o modifica una risorsa (idempotente)
- **DELETE:** elimina una risorsa identificata dall'URL
- **CONNECT:** richiede un accesso diretto attraverso TCP
- **OPTIONS:** richiede i metodi accettati per una determinata risorsa
- **TRACE:** si ottiene ciò che ha ottenuto il server dalla nostra richiesta

Ecco un esempio di messaggio che potresti ricevere in risposta ad una richiesta GET:

HTTP/1.1 200 OK

Codice di stato

Date: Tue, 14 Nov 2023, 12:10:23 GMT

Server:

Content-Type: text/html

Content-Length: 1234

Header

<html>

<head>

<title>Titolo della pagina</title>

</head>

<body>

<h1>Benvenuto nella mia pagina!</h1>

<p>Questo è un esempio di risposta a una richiesta GET.</p>

</body>

</html>

Body

Il messaggio inizia con la versione del protocollo HTTP utilizzata e lo stato della risposta (in questo caso, OK). Successivamente abbiamo l'header, in cui vengono specificati il tipo di contenuto della risposta (text/html) e la lunghezza del contenuto (1234 byte) e altre informazioni. Infine, il corpo del messaggio contiene il codice HTML della pagina richiesta.

Codici di stato:

- **1XX: informativa (codici di negoziazione tra client e server)**
- **2XX: codici di Successo della richiesta**
- **3XX: Ridirezione (errata specificazione della risorsa)**
- **4XX: Errore del client, la richiesta è errata**
- **5XX: Errore del server**

Informazioni dell'HEADER, Risposta Server:

- Server, nome del server
- Date, data e ora dell'invio della risposta
- Content-Length, dimensione in byte del file
- Content-Type, il tipo MIME (internet media type) del body (esempio: text/html)

ce ne sono altri a pagina 28 del libro di testo

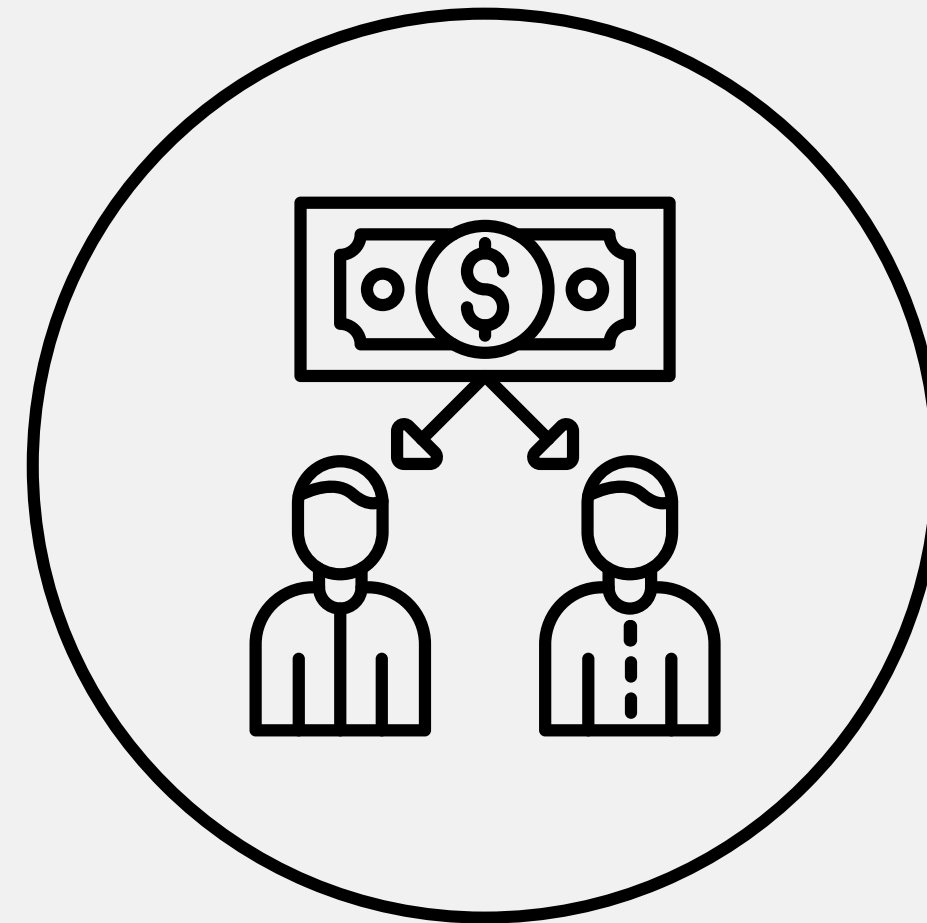
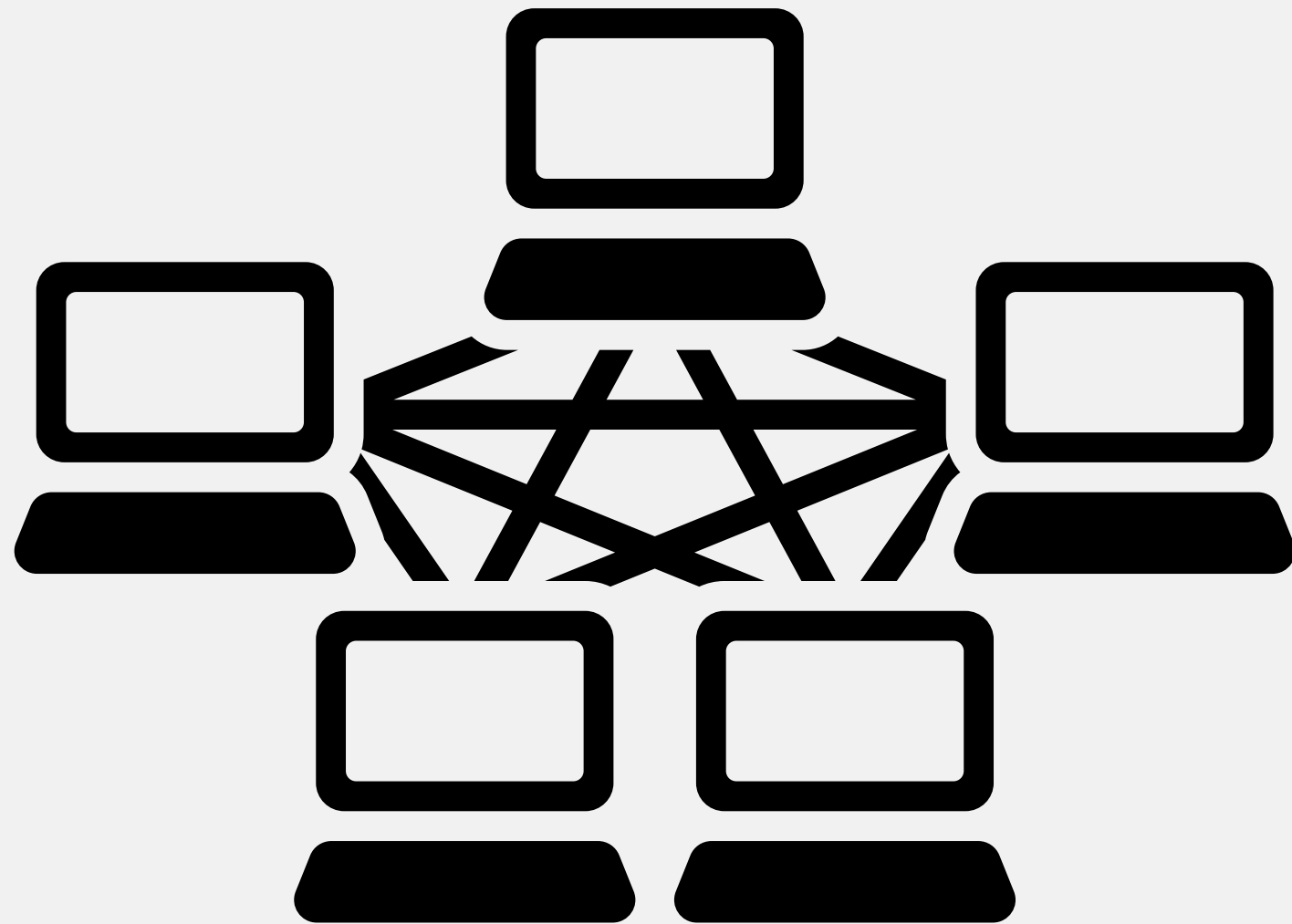
HTTPS come versione sicura di HTTP:

HTTPS è la versione sicura di HTTP, dove le comunicazioni sono criptate utilizzando il protocollo SSL/TLS. Questo previene l'intercettazione e la manipolazione dei dati trasmessi.

Uso di SSL/TLS per criptare la comunicazione: SSL (Secure Sockets Layer) e TLS (Transport Layer Security) sono protocolli crittografici che forniscono comunicazioni sicure su una rete informatica.

Benefici in termini di sicurezza e integrità dei dati: HTTPS protegge l'utente da numerosi attacchi informatici, garantendo che i dati trasmessi siano sicuri e non possano essere facilmente letti o modificati da terzi non autorizzati.

PEER to PEER



PEER TO PEER

Il modello peer-to-peer è un tipo di architettura di rete in cui ciascun partecipante (peer) funge sia da client che da server, condividendo risorse e responsabilità. A differenza dei modelli client-server, non esiste un server centrale; ogni peer è autonomo e contribuisce alla rete.

Utilizzo: Questo modello è comunemente usato per applicazioni come la condivisione di file (es. BitTorrent), streaming di contenuti e sistemi di comunicazione. La sua natura decentralizzata lo rende particolarmente adatto per queste applicazioni.

Caratteristiche

Decentralizzazione: In una rete P2P, non esiste un punto centrale di controllo o distribuzione delle risorse, il che la rende meno vulnerabile a guasti o attacchi su un singolo server.

Scalabilità: Le reti P2P possono facilmente scalare per accogliere più utenti, in quanto ogni nuovo peer aggiunto contribuisce con risorse aggiuntive alla rete.

Affidabilità e resilienza: La mancanza di un punto centrale di guasto rende le reti P2P più affidabili. Se un peer fallisce o si disconnette, gli altri possono continuare a operare senza interruzioni.

APPLICAZIONI:

Condivisione di file: BitTorrent è un esempio noto di P2P utilizzato per la condivisione di file. I file vengono suddivisi in parti che vengono condivise tra i peer, permettendo download efficienti e distribuiti.

Criptovalute: La tecnologia blockchain, come quella usata per Bitcoin, è un esempio di P2P nel contesto finanziario. Consente transazioni sicure e decentralizzate senza un'autorità centrale.

Comunicazione: Skype, nelle sue prime versioni, utilizzava una rete P2P per connettere gli utenti direttamente, migliorando l'efficienza delle chiamate.